

Space Complexity

CSE 211 (Theory of Computation)

Atif Hasan Rahman

Assistant Professor
Department of Computer Science and Engineering
Bangladesh University of Engineering & Technology

Adapted from slides by
Dr. Muhammad Masroor Ali

Space Complexity

- We consider the complexity of computational problems in terms of the amount of space, or memory, that they require
- We look at a class of problems that includes all of NP, and appears to include more
- This class is defined by allowing a Turing machine to use an amount of space that is polynomial in the size of its input, no matter how much time it uses
 - Initially, we shall distinguish between the languages accepted by deterministic and nondeterministic TMs with a polynomial space bound
 - We shall see that these two classes of languages are the same

Space Complexity

Space Complexity

Let M be a deterministic Turing machine that halts on all inputs. The **space complexity** of M is the function $f : N \rightarrow N$, where $f(n)$ is the maximum number of tape cells that M scans on any input of length n . If the space complexity of M is $f(n)$, we also say that M runs in space $f(n)$.

Space Complexity (Nondeterministic)

If M is a nondeterministic Turing machine wherein all branches halt on all inputs, we define its space complexity $f(n)$ to be the maximum number of tape cells that M scans on any branch of its computation for any input of length n .

Deterministic and Nondeterministic Space

SPACE and NSPACE

Let $f : N \rightarrow R^+$ be a function. The space complexity classes, $SPACE(f(n))$ and $NSPACE(f(n))$, are defined as follows.

- $SPACE(f(n)) = \{L \mid L \text{ is a language decided by an } O(f(n)) \text{ space deterministic Turing machine}\}$
- $NSPACE(f(n)) = \{L \mid L \text{ is a language decided by an } O(f(n)) \text{ space nondeterministic Turing machine}\}$

Deterministic and Nondeterministic Polynomial Space

PSPACE and NPSPACE

The space complexity classes, PSPACE and NPSPACE, are defined as follows.

- $\text{PSPACE} = \{L \mid L \text{ is a language decided by a polynomial space deterministic Turing machine}\}$
- $\text{NPSPACE} = \{L \mid L \text{ is a language decided by a polynomial space nondeterministic Turing machine}\}$

Example

- The NP-complete problem SAT can be solved with a linear space algorithm
- Space appears to be more powerful than time because space can be reused, whereas time cannot

M_1 = “On input $\langle \phi \rangle$, where ϕ is a Boolean formula:

1. For each truth assignment to the variables $x_1, \dots, x_m$ of ϕ :
2. Evaluate ϕ on that truth assignment.
3. If ϕ ever evaluated to 1, *accept*; if not, *reject*.”

Relationship among Complexity Classes

- $\mathcal{P} \subseteq \text{PSPACE}$ and $\mathcal{NP} \subseteq \text{NPSPACE}$.
 - The reason is that if a TM makes only a polynomial number of moves, then it uses no more than a polynomial number of cells in particular, it cannot visit more cells than one plus the number of moves it makes
- We will show that $\text{PSPACE} = \text{NPSPACE}$
- Overall relationship is $\mathcal{P} \subseteq \mathcal{NP} \subseteq \text{PSPACE}$

Time and Space Complexity

Theorem

If M is a polynomial-space bounded TM (deterministic or nondeterministic) and $p(n)$ is its polynomial space bound, then there is a constant c such that if M accepts its input w of length n , it does so within $c^{1+p(n)}$ moves.

PROOF: The essential idea is that M must repeat an ID before making more than $c^{1+p(n)}$ moves.

Proof

- c must exist because there are a limited number of ID's if the space used by the TM is limited
- Let t be the number of tape symbols of M and let s be the number of states of M
- Then the number of different ID's of M when only $p(n)$ tape cells are used is at most $sp(n)t^{p(n)}$
- That is we can choose one of the s states, place the head at any of $p(n)$ tape positions and fill the $p(n)$ cells with any of $t^{p(n)}$ sequences of tape symbols
- Pick $c = s + t$. Then consider the binomial expansion of $(t + s)^{1+p(n)}$ which is

$$t^{1+p(n)} + (1 + p(n))st^{p(n)} + \dots$$

- The second term is at least as large as $sp(n)t^{p(n)}$

Time and Space Complexity

Theorem

If L is a language in PSPACE (respectively NPSPACE) then L is accepted by a polynomial space bounded deterministic (respectively nondeterministic) TM that halts after making at most $c^{q(n)}$ moves, for some polynomial $q(n)$ and constant $c > 1$.

Proof Idea: Keep track of the number of moves. If TM has not accepted within $c^{1+p(n)}$ moves, halt.

Deterministic and Nondeterministic Polynomial Space

Theorem (Savitch's theorem)

For any function $f : N \rightarrow R^+$, where $f(n) \geq n$
 $\text{NSPACE}(f(n)) \subseteq \text{SPACE}(f^2(n))$ that is $\text{PSPACE} = \text{NPSPACE}$

PROOF: It is obvious that $\text{PSPACE} \subseteq \text{NPSPACE}$ since every DTM is technically a NTM as well. Thus we need only to show that $\text{NPSPACE} \subseteq \text{PSPACE}$.

Proof of Savitch's theorem

- We need to simulate an $f(n)$ space NTM deterministically.
- A naive approach is to proceed by trying all the branches of the NTM's computation, one by one
- The simulation needs to keep track of which branch it is currently trying so that it is able to go on to the next one
- But a branch that uses $f(n)$ space may run for $c^{O(f(n))}$ steps and each step may be a nondeterministic choice
- Exploring the branches sequentially would require recording all the choices used on a particular branch in order to be able to find the next branch.
- Therefore, this approach may use exponential space, exceeding our goal of $O(f^2(n))$ space.

Proof of Savitch's theorem

- Instead, we take a different approach by considering the following more general problem
- We are given two configurations of the NTM, I and J , together with a number m , and we test whether the NTM can get from I to J within m steps using only $f(n)$ space. We call this problem the **reachability problem**
- A DTM systematically tries all middle IDs K to check whether I can become K in $m/2$ moves, and then K can become J in $m/2$ moves

Proof of Savitch's theorem

```
BOOLEAN FUNCTION reach(I,J,m)
  ID: I,J; INT: m;
  BEGIN
    IF (m == 1) THEN /* basis */ BEGIN
      test if I == J or I can become J after one move;
      RETURN TRUE if so, FALSE if not;
    END;
    ELSE /* inductive part */ BEGIN
      FOR each possible ID K DO
        IF (reach(I,K,m/2) AND reach(K,J,m/2)) THEN
          RETURN TRUE;
        RETURN FALSE;
      END;
    END;
  END;
```

Proof of Savitch's theorem

- Observe that, although reach calls itself twice, it makes those calls in sequence, and therefore, only one of the calls is active at a time
- That is, if we start with a stack frame (I_1, J_1, m) , then at any time there is only one call $(I_2, J_2, m/2)$ one call $(I_3, J_3, m/4)$, another $(I_4, J_4, m/8)$ and so on, until at some point the third argument becomes 1
- If started with a move count of m , there can only be $\log_2(m)$ stack frames on the tape at any one time

Proof of Savitch's theorem

Given input w of length n , we discover what N does with input w by repeatedly placing the triple (I_0, J, m) on its tape and calling reach with these arguments where

- I_0 is the initial ID of N with input w
- J is any accepting ID that uses at most $p(n)$ tape cells.
The different J s are enumerated systematically
- $m = c^{1+f(n)}$

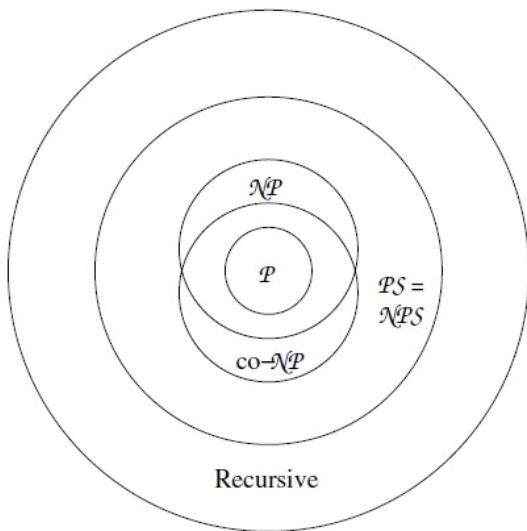
Proof of Savitch's theorem

- Now if N is an $f(n)$ space NTM that accepts, it does so within $c^{1+f(n)}$ steps for some constant c . So, $m = c^{1+f(n)}$
- Thus, there are no more than $\log_2(c^{1+f(n)})$ stack frames on the stack, which is $O(f(n))$
- Further, the stack frames themselves take $O(f(n))$ space. The reason is that the two IDs each require only $1 + f(n)$ cells to write down, and if we write m in binary, it requires $\log_2(c^{1+f(n)})$ cells, which is $O(f(n))$
- Thus, the entire stack frame, consisting of two IDs and an integer, takes $O(f(n))$ space

Proof of Savitch's theorem

- Since we have $O(f(n))$ stack frames at most, the total amount of space used is $O(f^2(n))$.
- This amount of space is a polynomial if $f(n)$ is polynomial, so we conclude that L has a DTM that is polynomial space bounded.

Conjectured Relation among Classes of Languages



Conjectured Relation among Classes of Languages

